Assignment 2: Dates Generator Report [GitHub_Repo]

Name: Sama Mohamed Abdelhamid **ID:** 202201867

1. Problem Formulation

1.1 Data Representation & Tokenization

The objective is a conditioned generation task where inputs (conditions) map to an output sequence (date). A custom tokenization strategy was implemented to handle the distinct nature of the inputs and outputs:

- **Conditions (X):** Word-level tokenization. The 4 conditions ([DAY], [MONTH], [LEAP], [DECADE]) were mapped to integer IDs using a dynamically built vocabulary.
- **Dates (Y):** Character-level tokenization (0-9, -). To assist the model in predicting a logical sequence, the output strings were temporarily reordered from dd-mm-yyyy to yyyy-mm-dd during training. This ensures the model predicts the year immediately after observing the [DECADE] condition, establishing a stronger initial context before predicting the month and day. Special tokens (<SOS>, <EOS>, <PAD>) were added to control sequence generation and handle batching.

1.2 Evaluation Metric

Standard classification accuracy is mathematically invalid for this task because there is a one-to-many relationship between conditions and correct dates. Instead, a custom ConditionValidator script was implemented. This deterministic algorithm tracks the "Compliance Rate" by checking each generated sequence to calculate the percentage of dates that successfully satisfy all four independent constraints (Day, Month, Leap Year, Decade), alongside strict calendar validity.

2. Architectures & Loss Functions

Four unique architectures were implemented to explore different generative paradigms.

- **Conditional VAE (CVAE):**
  - **Architecture:** An LSTM-based Encoder compresses the date and conditions into a latent space (mu, sigma). An LSTM Decoder utilizes the sampled latent vector and the conditions to autoregressively generate the sequence.
  - **Loss:** Cross-Entropy Loss (reconstruction) + KL Divergence (latent regularization). A beta weight of 0.1 was applied to the KLD to prevent posterior collapse.
- **Conditional GAN (CGAN):**
  - **Architecture:** The Generator maps latent noise and conditions to sequence logits using an LSTM. The Discriminator uses a Bidirectional LSTM to classify sequences as real or fake.
  - **Loss:** Binary Cross Entropy with Logits Loss. Gumbel-Softmax was utilized in the Generator to allow continuous backpropagation through discrete character sampling.

- **Date Transformer (Autoregressive):**
  - **Architecture:** A decoder-only style Transformer (2 layers, 4 heads). The encoded conditions act as a "prefix prompt," concatenated to the date sequence embeddings alongside Positional Encoding.
  - **Loss:** Cross-Entropy Loss, with padding tokens explicitly masked/ignored.
- **MLP Baseline:**
  - **Architecture:** A standard Feed-Forward network that flattens the 4 condition embeddings and projects them directly to the full target sequence shape through dense layers and Dropout.
  - **Loss:** Cross-Entropy Loss.

3. Output Analysis & Reflection

After running inference across all four trained architectures, the results highlight the steep difficulty of non-linear conditional generation for calendar math.

3.1 Transformer & MLP: Structural Mastery but Mode Collapse

Both the Transformer and MLP successfully learned the surface-level structures. They consistently generated the correct dd-mm-yyyy format, flawlessly mapped the [MONTH] condition to the correct month digit, and mapped the [DECADE] condition to the correct starting year digits.

- **Failure:** Both suffered severe mode collapse regarding the specific day. The Transformer overwhelmingly defaulted to generating the "24th" or "22nd" of the month (e.g., [WED] [JAN] [False] [180] -> 24-1-1802), while the MLP collapsed to generating the "1st" or "2nd". They optimized for the "easy" structural loss but failed to map the [DAY] condition to the calendar output.

3.2 VAE: High Variance but Calendar Hallucinations

The VAE successfully escaped the mode collapse seen in the Transformer and MLP. It generated a highly diverse set of days, learned the output formatting flawlessly, and correctly bound the months and decades (e.g., [MON] [JAN] [False] [190] -> 16-1-1909).

- **Failure:** While diverse, the VAE failed the strict calendar logic. It frequently mismatched the requested Day of the Week, and more notably, generated physically impossible dates. For example, given [WED] [FEB] [False] [189], it generated 30-2-1890 (February 30th does not exist).

3.3 CGAN: Catastrophic Sequence Failure

The CGAN entirely failed to generate valid sequences. Its output file consists of fragmented integers (233, 83355) and heavily malformed structures (83355135-0).

- **Failure:** Generating discrete sequences (text) with GANs is notoriously unstable. Standard GANs are designed for continuous data. Even with the Gumbel-Softmax trick, the gradients flowing from the Discriminator to the Generator remained too noisy, leading to a complete divergence within the 50 training epochs.

3.4 General Reflection

The core underlying issue is that the relationship between a specific date and its day of the week requires modular arithmetic (Zeller's congruence). Neural networks—especially simple models operating purely on character tokens—struggle to natively infer non-linear algorithmic math without explicit feature engineering. The models learned the visual patterns of dates but lacked the mathematical reasoning required to satisfy the [DAY] constraint simultaneously with the [LEAP] constraints.

4. Engineering Best Practices

To ensure robust training, replicability, and clean code structure, several best practices were enforced:

- **Teacher Forcing:** Utilized in both the VAE and Transformer to stabilize early autoregressive training.
- **Gradient Clipping:** Applied (max_norm=1.0) during recurrent network training to prevent exploding gradients.
- **Data Splitting:** The PyTorch DataLoader was paired with random_split to create a strict 90/10 train/validation split.
- **Modular Inference:** The predict.py script was designed with a command-line interface (argparse), allowing for dynamic model switching (via the -m flag) to individually test all four saved .pt weights without altering the underlying code.